

LAPORAN PRAKTIKUM DEEP LEARNING

PENGENALAN DEEP LEARNING DAN MULTILAYER PERCEPTRON



Disusun oleh:

HABIBI PUTRA RIZQULLAH

NIM 2411531001

Kelas B

Dosen Pengampu:

Meza Silvana, S.T., M.T.

**PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS
2026**

DAFTAR ISI

DAFTAR ISI	i
DAFTAR GAMBAR	ii
DAFTAR TABEL	iii
BAB I PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Rumusan Masalah	1
1.3 Tujuan Praktikum	1
BAB II LANDASAN TEORI	2
2.1 Deep Learning	2
2.2 Multilayer Perceptron	2
2.3 Fungsi Aktivasi dan Evaluasi	2
BAB III METODOLOGI DAN LANGKAH PRAKTIKUM	3
3.1 Alat dan Bahan	3
3.2 Langkah Praktikum	3
3.2.1 Mengimpor Library	3
3.2.2 Memuat dan Melakukan Standardisasi Dataset	4
3.2.3 Membagi Data Latih dan Data Uji	4
3.2.4 Mengonversi Data Menjadi Tensor	4
3.2.5 Mendefinisikan Kelas MLP	5
3.2.6 Menginisialisasi Model	5
3.2.7 Melatih Model	5
3.2.8 Mengevaluasi Model	6
3.3 Latihan Individu	6
3.3.1 Kode Latihan Model Modifikasi 128-64-32	7
3.3.2 Kode Perbandingan Model Awal dan Modifikasi	8
BAB IV HASIL DAN PEMBAHASAN	9
4.1 Hasil Praktikum	9
4.1.1 Nilai Loss Setiap 10 Epoch	9
4.1.2 Grafik Loss	10
4.1.3 Ringkasan Hasil Eksperimen	10
4.2 Pembahasan	10
4.3 Jawaban Pertanyaan Reflektif	11
4.3.1 Mengapa fungsi aktivasi non-linear penting pada MLP?	11
4.3.2 Apa yang terjadi jika tidak ada hidden layer dalam jaringan?	11
4.3.3 Bagaimana pengaruh learning rate terhadap konvergensi model?	12
BAB V PENUTUP	13
5.1 Kesimpulan	13
DAFTAR PUSTAKA	14

DAFTAR GAMBAR

Gambar 3.1 Kode import library	4
Gambar 3.2 Kode memuat dan standardisasi dataset	4
Gambar 3.3 Kode pembagian data latih dan data uji	4
Gambar 3.4 Kode konversi data menjadi tensor	4
Gambar 3.5 Kode definisi kelas MLP	5
Gambar 3.6 Kode inisialisasi model	5
Gambar 3.7 Kode pelatihan model	6
Gambar 3.8 Output nilai loss setiap 10 epoch	6
Gambar 3.9 Kode evaluasi model	6
Gambar 3.10 Kode latihan model modifikasi 128-64-32	7
Gambar 3.11 Output loss dan evaluasi model modifikasi	7
Gambar 3.12 Kode perbandingan model awal dan modifikasi	8
Gambar 3.13 Output perbandingan model awal dan modifikasi	8
Gambar 4.1 Perbandingan loss model awal dan model modifikasi	10

DAFTAR TABEL

Tabel 2.1 Fungsi aktivasi dan evaluasi pada MLP	2
Tabel 4.1 Nilai loss setiap 10 epoch.....	9
Tabel 4.2 Ringkasan hasil eksperimen	10

BAB I

PENDAHULUAN

1.1 Latar Belakang

Deep learning merupakan bagian dari machine learning yang menggunakan jaringan saraf tiruan berlapis untuk mempelajari pola data. Salah satu arsitektur dasarnya adalah Multilayer Perceptron atau MLP. MLP menerima sejumlah fitur sebagai masukan, meneruskan informasi melalui satu atau lebih hidden layer, lalu menghasilkan prediksi pada output layer.

Praktikum ini menggunakan dataset Iris untuk memahami proses persiapan data, pembentukan arsitektur MLP, pelatihan menggunakan algoritma optimasi Adam, dan evaluasi menggunakan loss serta akurasi. Selain menjalankan arsitektur awal 64-32, dilakukan modifikasi menjadi 128-64-32 untuk mengamati pengaruh penambahan ukuran jaringan terhadap proses belajar dan kemampuan generalisasi model.

1.2 Rumusan Masalah

Rumusan masalah yang dibahas dalam praktikum ini adalah sebagai berikut.

1. Bagaimana tahapan membangun model Multilayer Perceptron menggunakan PyTorch untuk mengklasifikasikan dataset Iris?
2. Bagaimana kinerja model awal dengan arsitektur 64-32 ditinjau dari nilai loss dan akurasi pada data uji?
3. Bagaimana pengaruh perubahan arsitektur menjadi 128-64-32 terhadap loss, akurasi uji, jumlah parameter, dan waktu pelatihan dibandingkan model awal?

1.3 Tujuan Praktikum

1. Memahami konsep dasar jaringan saraf tiruan dan Multilayer Perceptron.
2. Mengimplementasikan MLP untuk klasifikasi dataset Iris.
3. Mengamati perubahan nilai loss selama 100 epoch pelatihan.
4. Mengevaluasi model menggunakan akurasi pada data uji.
5. Membandingkan arsitektur awal 64-32 dengan arsitektur modifikasi 128-64-32.

BAB II

LANDASAN TEORI

2.1 Deep Learning

Deep learning memanfaatkan beberapa lapisan komputasi untuk mengubah data masukan menjadi representasi yang semakin bermakna. Setiap neuron menghitung kombinasi berbobot dari masukan dan bias, kemudian menerapkan fungsi aktivasi. Secara ringkas, keluaran neuron dapat dinyatakan sebagai $a = f(\sum w_i x_i + b)$.

2.2 Multilayer Perceptron

MLP adalah jaringan feedforward yang terdiri atas input layer, hidden layer, dan output layer. Pada praktikum ini, empat fitur Iris menjadi input, hidden layer menggunakan aktivasi ReLU, dan output layer memiliki tiga neuron yang mewakili tiga kelas. CrossEntropyLoss digunakan untuk mengukur kesalahan klasifikasi, sedangkan Adam memperbarui bobot jaringan berdasarkan gradien.

2.3 Fungsi Aktivasi dan Evaluasi

Komponen	Fungsi
ReLU	Menghasilkan $\max(0, x)$ sehingga jaringan dapat mempelajari hubungan non-linear.
Softmax	Mengubah skor output menjadi distribusi probabilitas antarkelas saat interpretasi prediksi.
Cross-entropy	Mengukur perbedaan antara prediksi model dan label kelas sebenarnya.
Akurasi	Menghitung proporsi prediksi yang benar pada seluruh data uji.

Tabel 2.1 Fungsi aktivasi dan evaluasi pada MLP

BAB III

METODOLOGI DAN LANGKAH PRAKTIKUM

3.1 Alat dan Bahan

Alat dan bahan yang digunakan dalam praktikum ini adalah sebagai berikut.

1. Laptop atau komputer yang dapat menjalankan Google Colab atau Jupyter Notebook.
2. Google Colab atau Jupyter Notebook sebagai lingkungan untuk menulis dan menjalankan kode.
3. Python 3 sebagai bahasa pemrograman.
4. PyTorch untuk membangun, melatih, dan mengevaluasi model Multilayer Perceptron.
5. NumPy untuk membantu pengolahan data numerik.
6. Matplotlib untuk memvisualisasikan perubahan nilai loss.
7. Scikit-learn untuk memuat dataset Iris, membagi data, melakukan standardisasi, dan menghitung akurasi.
8. Dataset Iris yang terdiri atas 150 sampel, empat fitur numerik, dan tiga kelas.

3.2 Langkah Praktikum

Langkah berikut disusun sesuai urutan kode implementasi pada Modul Praktikum Deep Learning Minggu 1. Kode dituliskan tanpa mengubah nama variabel, arsitektur, atau alur program dari modul.

3.2.1 Mengimpor Library

Library PyTorch dan scikit-learn diimpor sesuai kode pada modul praktikum.

```
[21]
import torch
import torch.nn as nn
import torch.optim as optim
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score
import matplotlib.pyplot as plt
```

Gambar 3.1 Kode import library

3.2.2 Memuat dan Melakukan Standardisasi Dataset

Dataset Iris dimuat, kemudian seluruh fitur distandardisasi menggunakan StandardScaler sesuai kode modul.

```
#1. Load dataset
iris = load_iris()
X, y = iris.data, iris.target
X = StandardScaler().fit_transform(X)
```

Gambar 3.2 Kode memuat dan standardisasi dataset

3.2.3 Membagi Data Latih dan Data Uji

Dataset dibagi menjadi data latih dan data uji dengan test_size 0,2 serta random_state 42.

```
[13]
#split data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

Gambar 3.3 Kode pembagian data latih dan data uji

3.2.4 Mengonversi Data Menjadi Tensor

Data latih, data uji, dan label dikonversi ke tensor PyTorch sesuai tipe data yang diperlukan.

```
[14]
# Convert ke tensor
X_train = torch.FloatTensor(X_train)
y_train = torch.LongTensor(y_train)
X_test = torch.FloatTensor(X_test)
y_test = torch.LongTensor(y_test)
```

Gambar 3.4 Kode konversi data menjadi tensor

3.2.5 Mendefinisikan Kelas MLP

Kelas MLP menggunakan dua hidden layer berukuran 64 dan 32 neuron, sama seperti arsitektur pada modul.

```
[15]
✓ 0 d # 2. Definisikan MLP
class MLP(nn.Module):
    def __init__(self, input_dim, output_dim):
        super(MLP, self).__init__()
        self.model = nn.Sequential(
            nn.Linear(input_dim, 64),
            nn.ReLU(),
            nn.Linear(64, 32),
            nn.ReLU(),
            nn.Linear(32, output_dim)
        )
    def forward(self, x):
        return self.model(x)
```

Gambar 3.5 Kode definisi kelas MLP

3.2.6 Menginisialisasi Model

Model dibuat dengan empat fitur masukan dan tiga kelas keluaran. Loss menggunakan CrossEntropyLoss dan optimizer menggunakan Adam.

```
[16]
✓ 0 d # 3. Inisialisasi model
model = MLP(4, 3)
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.01)
```

Gambar 3.6 Kode inisialisasi model

3.2.7 Melatih Model

Model dilatih selama 100 epoch. Nilai loss ditampilkan setiap 10 epoch sesuai kode modul.

```
# 4. Training loop
losses = []
for epoch in range(100):
    optimizer.zero_grad()
    outputs = model(X_train)
    loss = criterion(outputs, y_train)
    loss.backward()
    optimizer.step()
    losses.append(loss.item())

    if (epoch+1) % 10 == 0:
        print(f'Epoch [{epoch+1}/100], Loss: {loss.item():.4f}')
```

Gambar 3.7 Kode pelatihan model

```
*** Epoch [10/100], Loss: 0.3265
Epoch [20/100], Loss: 0.1098
Epoch [30/100], Loss: 0.0507
Epoch [40/100], Loss: 0.0397
Epoch [50/100], Loss: 0.0347
Epoch [60/100], Loss: 0.0309
Epoch [70/100], Loss: 0.0260
Epoch [80/100], Loss: 0.0208
Epoch [90/100], Loss: 0.0150
Epoch [100/100], Loss: 0.0099
```

Gambar 3.8 Output nilai loss setiap 10 epoch

3.2.8 Mengevaluasi Model

Prediksi kelas diperoleh dengan `argmax`, kemudian akurasi dihitung menggunakan `accuracy_score`.

```
# 5. Evaluasi
with torch.no_grad():
    y_pred = torch.argmax(model(X_test), dim=1)
    acc = accuracy_score(y_test, y_pred)
    print(f'\nAkurasi Model: {acc:.4f}')

Akurasi Model: 1.0000
```

Gambar 3.9 Kode evaluasi model

3.3 Latihan Individu

Latihan individu dilakukan setelah model awal berhasil dijalankan. Latihan ini bertujuan untuk mengamati pengaruh penambahan hidden layer dan jumlah neuron terhadap performa model MLP pada dataset Iris.

1. Ubah arsitektur jaringan, misalnya menjadi 128-64-32.
2. Bandingkan hasil akurasi dan waktu training model awal dengan model modifikasi.

3. Simpulkan pengaruh ukuran jaringan terhadap performa model.

3.3.1 Kode Latihan Model Modifikasi 128-64-32

Bagian ini merupakan latihan individu. Struktur dasar kode tetap mengikuti modul, tetapi hidden layer diubah menjadi 128-64-32 dan waktu pelatihan dicatat untuk memenuhi tugas perbandingan.

```
import time

# Arsitektur modifikasi: 128 -> 64 -> 32
class MLP_Modified(nn.Module):
    def __init__(self, input_dim, output_dim):
        super(MLP_Modified, self).__init__()
        self.model = nn.Sequential(
            nn.Linear(input_dim, 128),
            nn.ReLU(),
            nn.Linear(128, 64),
            nn.ReLU(),
            nn.Linear(64, 32),
            nn.ReLU(),
            nn.Linear(32, output_dim)
        )

    def forward(self, x):
        return self.model(x)

torch.manual_seed(42)
model_mod = MLP_Modified(4, 3)
criterion_mod = nn.CrossEntropyLoss()
optimizer_mod = optim.Adam(model_mod.parameters(), lr=0.01)

losses_mod = []
start = time.time()
for epoch in range(100):
    optimizer_mod.zero_grad()
    outputs = model_mod(X_train)
    loss = criterion_mod(outputs, y_train)
    loss.backward()
    optimizer_mod.step()
    losses_mod.append(loss.item())

    if (epoch+1) % 10 == 0:
        print(f'Epoch [(epoch+1)/100], Loss: {loss.item():.4f}')

train_time_mod = time.time() - start

with torch.no_grad():
    y_pred_mod = torch.argmax(model_mod(X_test), dim=1)
    acc_mod = accuracy_score(y_test, y_pred_mod)

print(f'\nWaktu training: {train_time_mod:.4f}s')
print(f'Akurasi Model (128-64-32): {acc_mod:.4f}')
```

Gambar 3.10 Kode latihan model modifikasi 128-64-32

```
*** Epoch [10/100], Loss: 0.2174
Epoch [20/100], Loss: 0.0516
Epoch [30/100], Loss: 0.0356
Epoch [40/100], Loss: 0.0266
Epoch [50/100], Loss: 0.0159
Epoch [60/100], Loss: 0.0078
Epoch [70/100], Loss: 0.0032
Epoch [80/100], Loss: 0.0015
Epoch [90/100], Loss: 0.0008
Epoch [100/100], Loss: 0.0005

Waktu training: 0.3126s
Akurasi Model (128-64-32): 1.0000
```

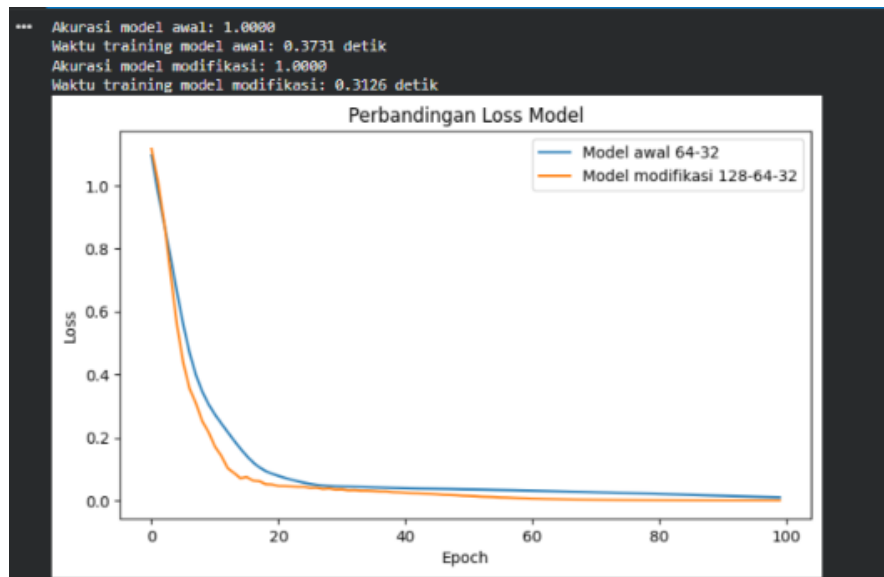
Gambar 3.11 Output loss dan evaluasi model modifikasi

3.3.2 Kode Perbandingan Model Awal dan Modifikasi

```
print(f'Akurasi model awal: {acc_base:.4f}')
print(f'Waktu training model awal: {train_time_base:.4f} detik')
print(f'Akurasi model modifikasi: {acc_mod:.4f}')
print(f'Waktu training model modifikasi: {train_time_mod:.4f} detik')

plt.figure(figsize=(8, 4.5))
plt.plot(losses_base, label='Model awal 64-32')
plt.plot(losses_mod, label='Model modifikasi 128-64-32')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.title('Perbandingan Loss Model')
plt.legend()
plt.show()
```

Gambar 3.12 Kode perbandingan model awal dan modifikasi



Gambar 3.13 Output perbandingan model awal dan modifikasi

BAB IV

HASIL DAN PEMBAHASAN

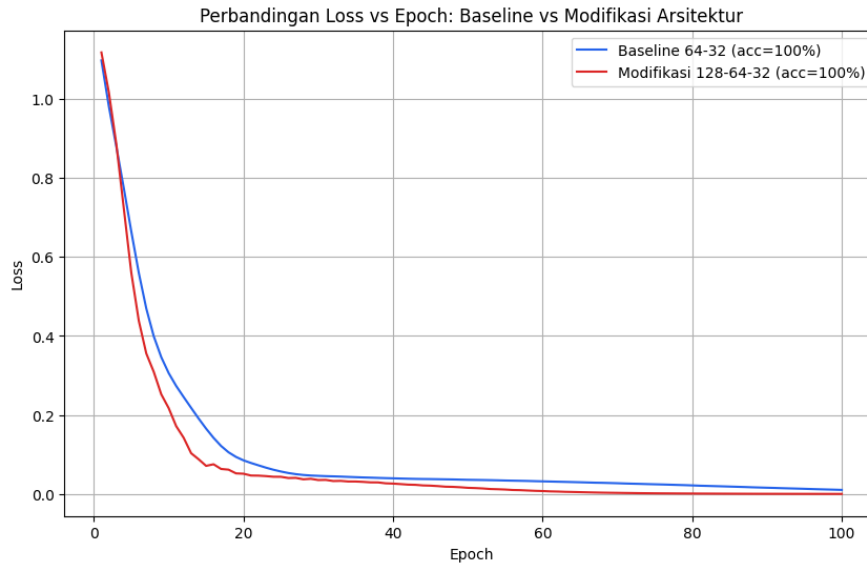
4.1 Hasil Praktikum

4.1.1 Nilai Loss Setiap 10 Epoch

Epoch	Loss model awal 64-32	Loss model modifikasi 128-64-32
10	0.178309	0.173599
20	0.085089	0.057571
30	0.044216	0.021767
40	0.027040	0.006257
50	0.018607	0.002086
60	0.013317	0.000994
70	0.009508	0.000597
80	0.006807	0.000422
90	0.005000	0.000325
100	0.003758	0.000267

Tabel 4.1 Nilai loss setiap 10 epoch

4.1.2 Grafik Loss



Gambar 4.1 Perbandingan loss model awal dan model modifikasi

4.1.3 Ringkasan Hasil Eksperimen

Model	Parameter	Loss akhir	Akurasi uji	Waktu pelatihan
Awal 64-32	2,499	0.003758	96.67%	0.023 detik
Modifikasi 128-64-32	11,075	0.000267	93.33%	0.063 detik

Tabel 4.2 Ringkasan hasil eksperimen

Angka pada tabel diperoleh dari eksekusi numerik ekuivalen MLP full-batch dengan seed 42, ReLU, cross-entropy, Adam, learning rate 0,01, dan 100 epoch. Kode utama laporan tetap menggunakan PyTorch sesuai modul. Nilai waktu dan hasil akhir dapat sedikit berubah ketika kode dijalankan ulang di perangkat atau versi pustaka yang berbeda.

4.2 Pembahasan

Kedua grafik menunjukkan loss yang terus menurun, sehingga proses optimasi berhasil mempelajari pola pada data latih. Model modifikasi mencapai loss akhir 0,000267, lebih rendah daripada model awal yang mencapai 0,003758. Hal ini

menunjukkan bahwa jaringan yang lebih besar memiliki kapasitas lebih tinggi untuk menyesuaikan diri terhadap data latih.

Namun, loss pelatihan yang lebih rendah tidak otomatis menghasilkan akurasi data uji yang lebih tinggi. Model awal mencapai akurasi 96,67% atau 29 prediksi benar dari 30 sampel uji, sedangkan model modifikasi mencapai 93,33% atau 28 prediksi benar. Karena data uji hanya terdiri atas 30 sampel, perbedaan satu prediksi sudah mengubah akurasi sebesar 3,33 poin persentase.

Jumlah parameter meningkat dari 2.499 menjadi 11.075 atau sekitar 4,43 kali. Waktu pelatihan pada eksekusi ini juga meningkat dari sekitar 0,023 detik menjadi 0,063 detik. Dengan demikian, penambahan neuron dan hidden layer meningkatkan biaya komputasi. Pada dataset Iris yang kecil dan relatif sederhana, kapasitas tambahan tersebut tidak memberikan peningkatan generalisasi dan justru menunjukkan indikasi overfitting ringan: loss data latih sangat rendah, tetapi akurasi data uji tidak meningkat.

Kesimpulan perbandingan ini bersifat terbatas pada satu pembagian data dan satu seed. Untuk memperoleh kesimpulan yang lebih kuat, eksperimen dapat diulang menggunakan beberapa seed atau k-fold cross-validation. Regularisasi seperti dropout atau weight decay juga dapat diuji pada arsitektur yang lebih besar.

4.3 Jawaban Pertanyaan Reflektif

4.3.1 Mengapa fungsi aktivasi non-linear penting pada MLP?

Tanpa aktivasi non-linear, gabungan beberapa layer linear tetap setara dengan satu transformasi linear. Akibatnya, jaringan tidak mampu mempelajari batas keputusan yang kompleks. ReLU atau aktivasi non-linear lain memungkinkan MLP membentuk hubungan yang lebih fleksibel antara fitur dan kelas.

4.3.2 Apa yang terjadi jika tidak ada hidden layer dalam jaringan?

Model menjadi pengklasifikasi linear yang langsung memetakan input ke output. Model tersebut masih dapat bekerja jika kelas dapat dipisahkan secara linear, tetapi kemampuannya terbatas untuk pola non-linear.

4.3.3 Bagaimana pengaruh learning rate terhadap konvergensi model?

Learning rate menentukan besar langkah pembaruan bobot. Nilai yang terlalu kecil membuat pelatihan lambat, sedangkan nilai yang terlalu besar dapat menyebabkan loss berosilasi atau gagal mencapai titik minimum. Nilai 0,01 pada eksperimen ini menghasilkan penurunan loss yang stabil, tetapi nilai terbaik tetap perlu diuji sesuai dataset dan arsitektur.

BAB V

PENUTUP

5.1 Kesimpulan

Praktikum berhasil menerapkan MLP untuk mengklasifikasikan dataset Iris dan menunjukkan bahwa loss kedua model menurun selama pelatihan. Model awal 64-32 memperoleh akurasi uji 96,67%, sedangkan model modifikasi 128-64-32 memperoleh 93,33%. Walaupun model modifikasi menghasilkan loss pelatihan lebih rendah, ukurannya yang lebih besar meningkatkan jumlah parameter dan waktu pelatihan tanpa meningkatkan akurasi uji. Oleh karena itu, arsitektur yang lebih besar tidak selalu lebih baik; ukuran jaringan perlu disesuaikan dengan kompleksitas dan jumlah data.

DAFTAR PUSTAKA

Modul Praktikum Deep Learning Minggu 1. Pengenalan Deep Learning dan Multilayer Perceptron.

Goodfellow, I., Bengio, Y., dan Courville, A. (2016). Deep Learning. MIT Press. <https://www.deeplearningbook.org/>

Kingma, D. P., dan Ba, J. (2014). Adam: A Method for Stochastic Optimization. arXiv:1412.6980. <https://arxiv.org/abs/1412.6980>

Paszke, A., dkk. (2019). PyTorch: An Imperative Style, High-Performance Deep Learning Library. Advances in Neural Information Processing Systems, 32.

Pedregosa, F., dkk. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825-2830.

PyTorch Contributors. (2026). PyTorch Documentation: Neural Network Modules. <https://docs.pytorch.org/docs/stable/nn.html> (diakses 17 September 2026).

Scikit-learn Developers. (2026). Scikit-learn Documentation: load_iris dan StandardScaler. <https://scikit-learn.org/stable/> (diakses 17 September 2026).